DiVA
http://www.diva-portal.org

Postprint

This is the accepted version of a paper presented at *International Conference on Product-Focused Software Process Improvement (PROFES 2023)*.

Citation for the original published paper:

Duszkiewicz, A G., Sørensen, J G., Johansen, N., Edison, H., Silva, T R. (2023)
Leveraging Historical Data to Support User Story Estimation
In: *Proceedings of International Conference on Product-Focused Software Process Improvement*
https://doi.org/10.1007/978-3-031-49266-2_20

N.B. When citing this work, cite the original published paper.

Permanent link to this version:
http://urn.kb.se/resolve?urn=urn:nbn:se:bth-25697

Leveraging Historical Data to Support User Story Estimation

Aleksander G. Duszkiewicz[1], Jacob G. Sørensen[2], Niclas Johansen[1], Henry Edison[3][0000−0002−9494−8059], and Thiago Rocha Silva[2][0000−0001−8961−4663]

[1] Morningtrain ApS, Odense, Denmark
{alek, nj}@morningtrain.dk
[2] The Maersk Mc-Kinney Moller Institute, University of Southern Denmark, Odense, Denmark
jacso18@student.sdu.dk, thiago@mmmi.sdu.dk
[3] Blekinge Institute of Technology, Karlskrona, Sweden
henry.edison@bth.se

Abstract. Accurate and reliable effort and cost estimation are still challenging for agile teams in the industry. It is argued that leveraging historical data regarding the actual time spent on similar past projects could be very helpful to support such an activity before companies embark upon a new project. In this paper, we investigate to what extent user story information retrieved from past projects can help developers estimate the effort needed to develop new similar projects. In close collaboration with a software development company, we applied design science and action research principles to develop and evaluate a tool that employs Natural Language Processing (NLP) algorithms to find past similar user stories and retrieve the actual time spent on them. The tool was then used to estimate a real project that was about to start in the company. A focus group with a team of six developers was conducted to evaluate the tool's efficacy in estimating similar projects. The results of the focus group with the developers revealed that the tool has the potential to complement the existing estimation process and help different interested parties in the company. Our results contribute both towards a new tool-supported approach to help user story estimation based on historical data and with our lessons learned on why, when, and where such a tool and the estimations provided may play a role in agile projects in the industry.

Keywords: User Stories · Agile Estimation · Natural Language Processing.

1 Introduction

Effort and cost estimation is still one of the critical activities of agile software development [26]. A fair amount of developers' time is spent on understanding, discussing, and establishing a consensus on an accurate estimate for user stories. Such activity is even costlier for companies that need to budget a software project

before gaining a development contract. In the agile context, the adoption of Planning Poker [15], despite being one of the most popular estimation techniques [27], remains a challenge for teams that lack the time, expertise, and domain knowledge to accurately estimate the user stories at hand, especially when there is no previous experience from similar tasks [16]. Moreover, estimates can easily be misjudged, and the development takes longer than first anticipated, leading to project loss.

Analogy-based estimation has been investigated for a long time as a strategy to overcome some of these challenges [24]. In the agile context, however, despite previous work has already demonstrated the benefits of Planning Poker to get better estimates [16,18,20], it is yet to know whether having access to the actual effort spent on similar user stories in the past could help developers to estimate user stories for new projects better. In practice, it is not trivial to identify similar stories from past projects, and many agile teams do not leverage this asset [3].

In this paper, we investigate to what extent user story information retrieved from a database of past agile projects can help developers estimate more easily the effort needed to develop new similar projects. Specifically, we seek to answer three research questions:

RQ1. How appropriate is the inference of user story similarity based on text similarity?

RQ2. To what extent can the actual effort spent on past user stories help developers estimate new similar user stories?

RQ3. What role an estimation tool can play to support the agile estimation process based on historical data?

To answer these questions, we followed design science principles to develop a software tool that employs Natural Language Processing (NLP) to identify and retrieve similar user stories from past projects. The tool was then evaluated in a focus group with key stakeholders. The contributions of this paper include the key findings of a focus group with an agile team that used the tool to estimate user stories for a new project in a real setting, as well as our lessons learned regarding the role of such a tool on agile projects in the industry.

We outline background and related work in Section 2 and our method in Section 3. In Section 4, we present our initial results, which are then discussed in Section 5 and concluded in Section 6.

2 Background and Related Work

As user stories are textual representations of features that need to be implemented, this work relies on text-based parameters to score story similarities. Text similarities can be literal or semantic. Literal similarity refers to looking at two strings and measuring how close or far from each other they are. Levenshtein distance, also called "edit distance" [31], is an example of such an algorithm. The output of the algorithm is the minimum number of character edits (replace, insert, or delete) to transform one string into another. The fewer edits, the more similar. Literal similarity, however, would only yield a high similarity score for

stories close to the same phrasing, meaning that many stories would never be caught. On the other hand, semantic similarity is a branch of NLP that goes beyond looking at the literal text. It instead utilizes machine learning to figure out similarities in text. For example, spaCy [1] is an open-source Python library that enables people who are not necessarily machine learning experts to conduct NLP tasks on words and sentences. spaCy comes with basic pre-trained models but can also be used with other models to achieve better scoring, such as the Universal Sentence Encoder (USE) [5] trained by Google. This allows for a deeper semantic analysis since, as long as the words carry the same meaning, two strings do not have to be worded the same way to achieve a high score.

The use of NLP to support Requirements Engineering (RE) activities is a long-term research topic for the RE community. For example, Zhao et al. [32] present a comprehensive literature review in this area. The identification of requirements similarities is one of the NLP applications on RE. Abbas et al. [2] have recently evaluated different NLP models to study the correlation between requirements similarity and software similarity. These studies ultimately aimed to allow for code reuse based on requirements similarities in past and current projects. Other authors have investigated requirements similarity to identify the relationship between requirements and software architecture [11], and also possible redundancies and inconsistencies in the requirements specification to improve the quality of requirements sentences [21].

Raharjana et al. [23] investigated how NLP techniques have been applied to requirements specified in the context of user stories. The authors argue that NLP studies on user stories are broad, ranging from discovering defects, generating software artifacts, identifying the key abstraction of user stories, and tracing links between model and user stories. In the context of agile planning, a study reported by Barbosa et al. [4] employs requirements similarity to suggest the existence of duplicate user stories in the product backlog. Many other previous works have reported using different NLP models to support the agile estimation process.

Choetkiertikul et al. [6] propose a prediction model named Deep-SE for estimating story points based on a combination of two deep learning architectures. According to the authors, their model learns from the team's previous story point estimates to predict the size of new issues. The authors expect the resultant estimates to be used in conjunction with existing estimation techniques practiced by the team. A different model is proposed by Fu & Tantithamthavorn [13] based on a transformer architecture. Their model, GPT2SP, differs from Deep-SE by explaining the suggested estimates. The authors also suggest the estimates provided by GPT2SP on a project can also be transferred to other projects. Results of a small survey conducted by the authors with agile practitioners suggested that explainable estimates are indeed more useful in this context. A good overview of contributions in this area is provided by Alsaadi & Saeedi [3] with a recent systematic literature review (SLR) on data-driven effort estimation techniques of agile user stories.

These contributions, however, are heavily focused on the performance of their respective prediction models. Besides providing a concrete software tool to support the agile estimation process (as opposed to prediction models), our contribution in this paper primarily differs from theirs by focusing on the qualitative data obtained in a real setting regarding the role that this kind of tool may play in agile projects in the industry. The findings of the SLR conducted by Alsaadi & Saeedi [3] also confirm the lack of studies focused on understanding the concrete usefulness of these solutions in real-world agile projects.

3 Research Methodology

Our overall research approach is oriented towards design science [29], which provides a concrete framework for dynamic validation in an industrial setting [14]. Design science is a problem-solving paradigm which creates artefacts to solve a problem in a certain context [29]. The artefacts may be represented in various structured forms, e.g. software, formal logic, method, technique, or conceptual structure [29]. Each problem is rooted in a certain context so that the artefacts can be designed to understand the context. Following the design science principle, our research method comprised a design task cycle: problem investigation, treatment design, and treatment validation.

3.1 Case Organization

Morningtrain ApS is a Danish medium-sized, full-service digital agency specializing in web design, programming, and online marketing. A large amount of developers' time in the company is currently spent on estimating the effort needed to complete projects, as each requirement has to be analyzed, refined, and estimated individually before a price can be proposed to the client. It is a critical phase. Even though a lot of resources are spent on this activity, there is no guarantee of success given the uncertainty of the estimation process. This research aimed at developing a solution that utilizes historical data to improve the estimation process, making it faster, more reliable, and more accurate.

3.2 Problem Investigation

We started investigating the problem by interviewing the company's head of development. During the interview, we were provided with a description of the current process in the company. Morningtrain has employed so far a weighted three-point estimation technique based on PERT [19] and would like to keep using it. It was also revealed that the company is currently trying to adopt Planning Poker more extensively and that estimates for user stories are mainly based on time rather than story points. We also did a workshop with the relevant stakeholders to get a complete picture of the problem. Open-ended questions were used to understand documents and processes currently in use. To elaborate on the process, the company shared its current requirements spreadsheet and

information on how this document is used on a daily basis for effort and cost estimation. It was revealed that the company would like to transfer this process to a new system, which should be able to generate a project on Jira based on the entered requirements, as well as provide historical data to support the process.

The way the company currently tracks effort is through its own internal hub with a tool where developers can register the time spent on a task. A Jira ID is provided during the process so the internal system matches the current Jira project and automatically updates the time-tracking field on that particular Jira issue. The time registered on previous Jira issues is expected to be used as ground truth effort estimates in the proposed solution. The proposed solution should also be able to support upfront planning and estimation at the beginning of the project, as well as ongoing estimation in each iteration, as they are both part of the challenges currently faced by the company. The ultimate purpose of the solution is to help reduce uncertainty as much as possible during the estimation process.

At the end of the aforementioned workshop, both functional and non-functional requirements for the system were identified. The non-functional requirements concerned maintainability and performance. The functional requirements included that the system should: *(i)* provide relevant historical data from similar user stories, *(ii)* keep the weighted three-point estimate technique, and *(iii)* be integrated with Jira providing a two-way communication of project data. Another particular characteristic identified in the company's process is that it deals with a lot of data when creating user stories for a new project. Part of this data is used as a customer's assurance that there is a mutual understanding of the features. Another part is used to describe the technical steps required to implement the stories, and another part describes the feature itself. Not all entries on a story follow a specific format or rule set. For example, the technical description for one story may include a detailed description of classes and patterns to follow, while for others, it may be completely empty. Most feature descriptions, however, follow the Connextra template for user stories [7].

3.3 Treatment Design

To address the investigated problems, we co-designed a solution with the company. The whole process, from the first interviews until the product evaluation, took seven months and consisted of three main steps. First, we conducted the interviews and workshops described above. Next, we did an informal literature review of the existing methodologies for scoring user stories, namely literal and semantic similarity. During this step, we also reviewed the existing tools and applications that had the potential to partially or fully address the problem.

Finally, the third step consisted of reviewing existing technologies based on the current set-up available in the company and implementing the tool according to the functional and non-functional requirements [12]. The tool employs semantic analysis to score semantic similarity among user stories. Semantic similarity analysis has been implemented using spaCy, which compares two strings

through its similarity function that uses cosine similarity to calculate a similarity score. While spaCy comes with a selection of standard models that can be directly used, when tested with our database, the similarity scores were mostly in the high end, regardless of the actual similarity. This made us opt for USE [5], a pre-trained model developed by Google. USE was trained with a large variety of text, which makes it a good choice for general-purpose tasks. During our tests, the similarity scores with USE were closer to what we expected based on a manual analysis of the stories in the database.

The system compares a new story against all other stories marked with the status of "done" in the database. During the comparison process, the user stories are pre-processed to remove the template skeleton (i.e., the sub-strings *"As a"*, *"I want"*, and *"So that"*) since we noticed this actually improves the parsing. We set our similarity threshold as 60% (i.e., any user story with a similarity score of 0.6 or above is returned to the user) as higher thresholds were not returning enough results in our pilot studies. After finding similar stories, we use the actual effort spent on the highest-scoring user story returned as the most likely estimate for a new similar story. Overall, the system workflow is not linked to any estimation process or technique in particular, so the tool could be used to support teams employing different estimation strategies.

3.4 Treatment Validation

Through action research, we proposed an intervention in the company by using the tool to estimate a new project that was about to start. We wanted to observe how the tool would affect the developers' work in a real-life context. After using the tool to estimate the user stories for a new project, the developers participated in a focus group to share their experiences. From conception to execution, the intervention took about a month. The new project to be estimated was a website to sell courses online. For the study, the tool's database was seeded with five past projects previously developed by the company. These projects included B2B systems, shipbroking services, subscription management, business websites, and services for distribution platforms. In total, 250 user stories have been used in the study. The study included 6 participants, as shown in Table 1.

After a brief introductory training about the tool, the participants were instructed to use it independently for a couple of days to explore the features and get estimates for the new user stories at hand. The focus group that followed was held on the company premises with the participation of two researchers as moderators. After signing an ethical agreement explaining the nature of the study and giving permission for the discussion to be recorded and anonymously transcribed, the participants answered a demographic questionnaire. During the session, the moderators asked for feedback on each feature, including the accuracy of the estimates provided, its usefulness, the choice of techniques employed, the adopted classification threshold, its performance compared to a manual approach, and the role and practical use of the tool on future projects. After the session, two researchers independently transcribed the recording, codified the main topics, and classified the findings by themes [9].

Table 1. Background information of the participants.

ID	Role	$\mathbf{E}^a$	$\mathbf{E}^b$	$\mathbf{E}^c$	$\mathbf{E}^d$
P1	Software Developer	1-3 years	1-3 years	$<$ 1 years	$<$ 1 years
P2	Software Developer	5-10 years	5-10 years	$<$ 1 years	1-3 years
P3	Software Developer	5-10 years	5-10 years	$<$ 1 years	$<$ 1 years
P4	Software Architect	5-10 years	1-3 years	$<$ 1 years	$<$ 1 years
P5	Software Developer	3-5 years	3-5 years	$<$ 1 years	$<$ 1 years
P6	Software Developer	5-10 years	5-10 years	1-3 years	$<$ 1 years

[a]Overall professional experience, [b]Experience in the current role, [c]Exp. in writing user stories, [d]Exp. in estimating user stories.

4 Results

In this section, we report the findings of our study. The findings are structured based on the research questions.

4.1 RQ1. How appropriate is the inference of user story similarity based on text similarity?

During the validation stage with the developers, they agreed that inferring user story similarity based only on text similarity is not enough, as user stories can be similar but might describe a whole different context.

> *"I don't think it's enough on its own. I think we need way more context, like tags, keywords, titles."* – P6

> *"It's a good way but it needs more ways to make sure that it's actually what we're looking for."* – P2

The participants also highlighted the fact that the tool returned few similar stories. They explicitly mentioned that lowering the similarity threshold to return more stories (even the less similar ones) would be more useful than returning just the most similar ones.

> *"I think the threshold might need to be lower because most of us didn't really find much, but I think rather than just have a threshold, you should also maybe just be shown the best matches, even if the numbers are only like 20 or 13. Because sometimes there might actually be something you can find. ... so you have like the list that kind of matches and then you go maybe this one actually is useful, OK then I can look at that one. So don't withhold information is what I'm trying to get at."* – P5

One of the participants suggested being more specific when writing user stories in the first place. Another participant, however, raised a concern that it would be hard to find similar user stories if they become too specific and oriented towards a specific project.

> *"It would be hard to find matches because you become specific about the project and the task, and then suddenly it'll be like, "oh but these don't match", and then we actually can't find anything."* – P5

Furthermore, there was a discussion about finding similar stories using keywords (tags), which extrapolates the use of tags we had previously anticipated while developing the tool (i.e., for pre-filtering stories) and later abandoned.

> *"Instead of having to write the user story as a whole, you could actually search using the keywords. So I want something to do with navigation, for example, I can just look up navigation, user stories written before."* – P1

> *"I think you might be on to something there because if you like to have repeat or whatever you just insert keywords you feel like and then you can actually use those to match up against other previous stories that have like maybe similar keywords...because maybe I'm like OK, I need to make I don't know "WordPress contact form seven", with like some fields and it needs to sync up, so I just write in "context form 7", "WordPress", "confirmation mail", etc. and then that'll already basically describe any similar task with the keywords alone."* – P5

Key Findings: Information should not be withheld. Even if no stories pass the threshold, the top-scoring stories should be displayed regardless. Alternative search techniques like tag search could supplement the textual similarity search in order to provide more context for the model, but also for the returned stories.

4.2 RQ2. To what extent can the actual effort spent on past user stories help developers estimate new similar user stories?

The participants also agreed that looking at the user story alone is not enough to judge whether the work required to implement such a story would be the same as the work required to implement another story deemed semantically similar by the model. An example of this came from two different participants, a back-end and a front-end developer. Both found the same past user story when trying to estimate a new similar one. The back-end developer agreed to the suggested most likely effort, while the front-end developer would have used much more time on it.

> *"I did the same like I found the same navigation user story and its estimates, and like as a front-end developer, I would have used a lot more time spent on that assignment. I think it is usually like 6 to 10 hours we use on that, and it was below 1...I'm not sure if it was like a back-end user story estimate or a front-end one"* – P3

The participants also found that the technologies involved with the implementation of different user stories have a large impact on the effort spent. Without the information about the technology stack on the historical user story, it is hard for the developers to judge whether or not an estimate is realistic.

> *"The big thing here for me is Laravel, is it WordPress? Because there's no similarity basically, in the implementation in the end, it would be completely useless to me knowing that it took five hours in WordPress if I'm doing it in Laravel."* – P6

> *"What is the actual stack for this particular project? Because the whole stack has something to say as well. If it's Blade there is one way to do it, if it's React, it's another way to do it, so it's very important to know the whole stack."* – P4

Key Findings: The context of the stories is crucial. An estimate cannot be reused if the tech stack is not similar or if the work is not done in the same part of the system, i.e., front-end or back-end.

4.3 RQ3. What role an estimation tool can play to support the agile estimation process based on historical data?

Even though the participants perceived the tool as being useful, they would rather use it alongside the Planning Poker sessions because these sessions help to bring context to an estimate.

> *"We would probably estimate faster because now we get more inputs on the actual time spent before on the previous task, which we can use to get the most accurate estimates, but I would probably not stop using Planning Poker"* – P1

> *"With Planning Poker we actually get like some face-to-face sparring, with the tool you basically automate sparring. You will get a number, but you don't really know why it received those numbers. With Planning Poker, we actually get to the reason why."* – P5

One of the participants disagreed with using the tool alongside the Planning Poker sessions.

> *"But I don't want to be biased by the system. My point is because if you do that, then the Planning Poker part is useless."* – P6

A participant also emphasised the importance of monitoring their own accuracy while estimating stories.

> *"I think I need to get feedback on the accuracy we are actually doing at the moment because if we do not get better at estimating by using the tool I am probably not seeing the use of the tool"* – P1

The participants also pointed out the need to expand the database of past projects as it would provide more stories and improve the estimation process.

> *"I think the only thing that's holding me back is not finding similar stories. I mean, I look at it I'm gonna write this story so I might as well copy and paste it into the tool and see what it gives me. But if it keeps giving me nothing, then it's obviously not useful, but if it gives me something at least have something to compare my thoughts with something else so I don't see why I wouldn't use it."* – P6

Regarding the Jira integration, participants suggested the tool should not only provide a two-way communication, but developers should also be able to add projects from Jira, in a plug-and-play setting.

> *"I think the way we would do it would probably be like adding the Jira board and the project manager doing all the setup and then like using this as a support tool to like log all the user stories and estimates coming over, so if it could be like a seamless sync where you can just plug and play on an existing project..."* - P3

The participants also had thoughts of who would benefit the most from the tool within the company, e.g., the Customer Care department. This department could search for related stories, receive the proposed estimate, and then consult it with a developer whether this would be a real effort to use.

> *"I actually see more usage for this tool in the Customer Care department. I think it would benefit having one or two Product Owners sitting with the tool and doing this for all the assignments they would get in the Customer Care department and then just dividing it out, and then we would more or less be the technical consultant on the estimates that we look at and see if this is anywhere near what we would think and such. So I think that would work well."* - P1

> *"If they (Customer Care department) could put in the user story and find the matching user story and then ask a developer to sign off on the estimate then yeah it would already have an estimate that was roughly written, but we could double-check it and see that this actually match, that could be very nice."* - P2

Key Findings: The tool could be used as support for the Planning Poker sessions, either before as preparation help or after to see if the estimates are off. A Jira plug-and-play setting would also be useful to constantly feed the system with new stories and estimates. The tool could also be used to provide rough estimates before developers can provide real ones. Finally, it has also a role as a support to help developers improve their own accuracy while estimating.

5 Discussion

This section discusses and makes sense of the findings of our study. It is composed of three main parts. The first part discusses the lessons learned, followed by the implications to practice. The third part discusses the limitations of this study.

5.1 Lessons Learned

One of the critical concerns raised by the developers is searching for similar user stories only based on textual similarity. They mentioned that even if they found a similar story, they would not be able to make a qualified decision on whether to use the actual effort, as they could not base it on an actual context. Here, it was mentioned that the stories should include more context, such as categorization, who has made the story, and the relationship with the requirements and projects. During the study, it became evident that the required effort cannot be compared even if two stories are similar if they were developed for different development environments. Thus, to improve on the concern, we will need to make sure that retrieved stories include more context to help the decision process during the estimation.

An important issue raised by the participants was the amount of data that would be required for the tool to start returning more similar user stories. This also has an impact on the accuracy of the user story identification. We hypothesize whether it would be valuable to train a model specifically for the purpose of finding similarities between user stories. We did not do this initially due to the very limited data available in the company to perform training on. While previous research shows that some datasets of user stories from industry exist (e.g., [10]), they are still in a very small number for properly training NLP models.

On another front, an interesting feature suggested by the developers was to include real-time feedback on how well they estimate new stories. Currently, the tool does not support any form of feedback as suggested by the participants. If the feature were to be implemented to make the tool more useful, it should be able to give feedback on whether the estimation is accurate. This could help the developers get better at estimating, which could also reduce the time spent on estimating new stories.

Another lesson learned refers to the Jira integration. Participants mentioned the tool should also be able to add projects from Jira. This suggests a seamless plug-in-play integration with Jira that would, over time, help improve the tool's usefulness as data can be added more easily. Such a feature would expand the integration currently provided by the tool and possibly raise the flow of user stories into the system, ultimately providing more stories for analysis.

There are also lessons learned regarding the role of the tool during the estimation process. Participants had conflicting views on how the tool should be used alongside Planning Poker. While some participants suggested it might replace the sparring that is normally observed during Planning Poker (since it would only require them to search for similar stories and use the actual effort

from a match), others stated that it would be more interesting to use the tool before the session, which could make the whole session useless. Other participants mentioned that they would not want to be biased by the tool, as they would like to have their thoughts on what estimate to give and not let the tool tell them exactly how long a story would take. This can be viewed both as a benefit and a drawback. On the one hand, the tool could be used to find a similar story, get the actual effort from that story, and then they could sign off on that estimate. On the other hand, it could also be a drawback since it removes the sparring that would normally be part of a Planning Poker session, where each developer could give his own estimate and explain their rationale. Thus, there might be some more considerations on improving the tool in this area to be more supportive and not completely remove the sparring the developers would normally have.

Lastly, the participants suggested this tool could play a role in different departments of the company. They explicitly mentioned that the Customer Care department would be a suitable place, as they suggested that the tool could support incoming change requests when a project is set into production. It was pointed out that the Customer Care team should also be able to use this tool to get a rough estimate based on past stories, and then they could consult the estimate with a developer who would sign off on that estimate. This suggests the tool has the potential to evolve to support more processes inside the company, which involve not only the start phases of the software development processes but also the last phases (maintenance and production) when new incoming change requests can be estimated more accurately with the help of the tool along with developer consultation.

5.2 Implications for Practice

Based on the results we discussed above, we provide a set of actionable recommendations for teams considering the use of automated estimates in their processes.

Lower thresholds (RQ1) Lower similarity thresholds are more useful as developers could still be able to match their own stories to the found ones and evaluate whether a higher or a lower percentage would be a match for them. In addition, not only stories marked as "done" should be shown, but also those that are ongoing because developers would still be able to find these stories useful even if they are not finished yet.

Context is paramount (RQ2) More context should be added to the retrieved stories as developers found it difficult to evaluate the actual effort used on a story without any context. It was suggested to implement a form of categorization that could help them better evaluate a story.

Keywords are important for search (RQ1) Searching similar stories based on specific keywords (or tags) would be helpful. Different methodologies could be

applied with keywords to search similar stories. For example, developers could search for a specific keyword, and then the tool would return stories related to that specific keyword. To further assist the search process, a user story field could be added for the developer to insert the user story that s/he is currently working on. Whenever the developer searches with a keyword and a specific user story, the NLP similarity process can be used as an addition to score these stories and return the best matching stories back to the developer.

Jira in a seamless, plug-and-play integration (RQ3) Currently, projects can be created inside the tool and transferred to Jira, where changes from both systems are reflected afterwards in two-way communication. However, the tool does not support the other way around, i.e., transferring projects from Jira to the tool. This feature would greatly help to accommodate changes, as project managers and Product Owners would no longer have any trouble adding stories by transferring Jira projects directly into the tool. Additionally, this seamless feature would also help with the concern regarding the lack of data.

Automated feedback on accuracy is a plus (RQ3) The tool should provide some feedback on how accurately developers estimate new stories. This would help them to get better at estimating, which would give greater value to the customer. The tool should give feedback throughout the process, i.e., from when developers receive the story after it has been estimated using the tool and finally after delivering the accomplished purpose of that story to the customer. Through this accuracy cycle, the tool should be able to show figures based on the story estimation process so the company can see this data as feedback on that process cycle.

5.3 Threats to Validity

Reliability Validity Reliability validity concerns the extent to which the data and analysis depend on the specific researchers [30]. Another researcher with more extensive knowledge and experience in design science research was also engaged in the review of the design and execution of the study. In addition, the focus group transcript used for the data analysis was sent back to the participants for review and shared among all authors. These practices helped to reduce biases during data collection and analysis.

Internal Validity The retrospective analysis nature of this study makes it vulnerable to historical types of internal validity threats [30]. A company representative with extensive knowledge of the user story estimation process helped identify the key processes and their efficacy. All focus group participants had extensive knowledge about the company's user story-based effort estimation process. However, our interview is still vulnerable to internal validity threats. We acknowledge that the number of participants is a limitation of this study.

Nonetheless, the vast years of experience that the participants have (i.e., four are about ten years) and the use of company documentation should reduce the threat to validity.

Construct Validity Construct validity refers to the extent to which the operational measures, e.g., the construct discussed in the focus group that is studied, represent the study's objective [30]. In this study, other researchers reviewed the questions before conducting the actual focus group to ensure that they were interpreted in the same way. In addition, data source triangulation was used to strengthen the evidence generated in this study.

External Validity External validity is concerned with the extent of generalizability [30] and transferability of a study [17]. The study presented here is based on five software development projects in a software company. Providing a detailed description of the context (see Section 3) helps in improving the study's external validity [22]. Even though each company and software development is unique, analytical induction helps to determine the generalizability between cases [28]. Hence, we provide an in-depth analysis of each case and carefully describe the context and provide clear insights into a particular context. Practitioners can easily compare the studied context with their own and take into consideration the findings and guidelines that we identified in their processes.

6 Conclusion

In this paper, we presented a study with a software tool that employs NLP to retrieve information from past user stories to support developers through the agile estimation process. The tool has been evaluated in a real setting with a team of six developers from the company estimating user stories for a real project that was about to start. Through the qualitative study, we found that textual similarity alone is not enough for developers to be able to make qualified decisions. The retrieved stories were missing more context, such as categorization, relationship with the referred project, and requirements. Once the developers have the contextual knowledge needed to fully understand a related user story, the actual effort spent on that story can indeed offer a baseline number for developers to use as a starting point for the new estimate. It is then up to them to make an informed decision based on the available data on whether or not the new story at hand would require more or less effort to implement. As the Planning Poker is part of the agile estimation process in the company, the developers will rather keep using it to estimate new projects. The tool has the potential to make the estimation process better and can be used either before or after the Planning Poker session. The tool is oriented towards complementing the sessions rather than replacing them. It can also guide developers on how accurate their estimates are based on the historical data available.

Future work could investigate cross-company historical data to address the low number of stories for NLP training. A partnership with companies around

the Atlassian software ecosystem could be a way ahead. An issue that arises, in this case, is data confidentiality, as it would potentially expose business-critical user story information to other companies. In the future, as the collection of data grows, we could also experiment with training our own model in order to check if there are any relevant gains in accuracy and volume of stories retrieved. Another possibility to achieve better results would be replacing USE with other pre-trained models, such as the Infersent model released by Facebook [8], which is also a general-purpose model. This would require additional research since not all models are supported out of the box in the spaCy library. Despite recent replication studies [25] having challenged the performance of specific models targeted at agile estimation, such as Deep-SE, it could also be worthwhile to investigate their performance within our approach.

Acknowledgments

This work was supported by the University of Southern Denmark's Internal Strategic Fund and ELLIIT: the Swedish Strategic Research Area in IT and Mobile Communications.

References

1. spacy (2021), `https://spacy.io`
2. Abbas, M., Ferrari, A., Shatnawi, A., Enoiu, E., Saadatmand, M., Sundmark, D.: On the relationship between similar requirements and similar software. Requirements Engineering pp. 1–25 (2022)
3. Alsaadi, B., Saeedi, K.: Data-driven effort estimation techniques of agile user stories: a systematic literature review. Artificial Intelligence Review pp. 1–32 (2022)
4. Barbosa, R., Silva, A., Moraes, R.: Use of similarity measure to suggest the existence of duplicate user stories in the srum process. In: 2016 46th Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshop (DSN-W). pp. 2–5. IEEE (2016)
5. Cer, D., Yang, Y., Kong, S., Hua, N., Limtiaco, N., John, R.S., Constant, N., Guajardo-Cespedes, M., Yuan, S., Tar, C., Sung, Y., Strope, B., Kurzweil, R.: Universal sentence encoder. CoRR **abs/1803.11175** (2018)
6. Choetkiertikul, M., Dam, H.K., Tran, T., Pham, T., Ghose, A., Menzies, T.: A deep learning model for estimating story points. IEEE Transactions on Software Engineering **45**(7), 637–656 (2018)
7. Cohn, M.: User stories applied: For agile software development. Addison-Wesley Professional (2004)
8. Conneau, A., Kiela, D., Schwenk, H., Barrault, L., Bordes, A.: Supervised learning of universal sentence representations from natural language inference data. CoRR **abs/1705.02364** (2017), `http://arxiv.org/abs/1705.02364`
9. Cruzes, D.S., Dyba, T.: Recommended steps for thematic synthesis in software engineering. In: 2011 international symposium on empirical software engineering and measurement. pp. 275–284. IEEE (2011)
10. Dalpiaz, F.: Requirements data sets (user stories) (2018), `https://data.mendeley.com/datasets/7zbk8zsd8y/1`

11. De Boer, R.C., Van Vliet, H.: On the similarity between requirements and architecture. Journal of Systems and Software **82**(3), 544–550 (2009)
12. Duszkiewicz, A.G., Sørensen, J.G., Johansen, N., Edison, H., Silva, T.R.: On identifying similar user stories to support agile estimation based on historical data. In: Agil-ISE@CAiSE. pp. 21–26 (2022)
13. Fu, M., Tantithamthavorn, C.: Gpt2sp: A transformer-based agile story point estimation approach. IEEE Transactions on Software Engineering (2022)
14. Gorschek, T., Garre, P., Larsson, S., Wohlin, C.: A model for technology transfer in practice. IEEE Software **23**(6), 88–95 (2006)
15. Grenning, J.: Planning poker or how to avoid analysis paralysis while release planning. Hawthorn Woods: Renaissance Software Consulting **3**, 22–23 (2002)
16. Haugen, N.C.: An empirical study of using planning poker for user story estimation. In: AGILE 2006 (AGILE'06). pp. 9–pp. IEEE (2006)
17. Lincoln, Y.S., Cuba, E.G.: Naturalistic Inquiry. Sage Publication (1985)
18. Mahnič, V., Hovelja, T.: On using planning poker for estimating user stories. Journal of Systems and Software **85**(9), 2086–2095 (2012)
19. Miller, R.W.: Schedule, cost, and profit control with pert: A comprehensive guide for program management (1963)
20. Moløkken-Østvold, K., Haugen, N.C., Benestad, H.C.: Using planning poker for combining expert estimates in software projects. Journal of Systems and Software **81**(12), 2106–2117 (2008)
21. Park, S., Kim, H., Ko, Y., Seo, J.: Implementation of an efficient requirements-analysis supporting system using similarity measure techniques. Information and Software Technology **42**(6), 429–438 (2000)
22. Petersen, K., Wohlin, C.: Context in industrial software engineering research. In: Proceedings of the 2009 3rd International Symposium on Empirical Software Engineering and Measurement. p. 401–404 (2009)
23. Raharjana, I.K., Siahaan, D., Fatichah, C.: User stories and natural language processing: A systematic literature review. IEEE Access **9**, 53811–53826 (2021)
24. Shepperd, M., Schofield, C.: Estimating software project effort using analogies. IEEE Transactions on software engineering **23**(11), 736–743 (1997)
25. Tawosi, V., Moussa, R., Sarro, F.: Agile effort estimation: Have we solved the problem yet? insights from a replication study. IEEE TSE **49**(4), 2677–2697 (2022)
26. Trendowicz, A., Jeffery, R.: Software Project Effort Estimation: Foundations and Best Practice Guidelines for Success. Springer Publishing (2014)
27. Usman, M., Mendes, E., Börstler, J.: Effort estimation in agile software development: a survey on the state of the practice. In: Proceedings of the 19th international conference on Evaluation and Assessment in Software Engineering. pp. 1–10 (2015)
28. Wieringa, R.J.: Case study research in information systems engineering. In: Proceedings of the 25th International Conference on Advanced Information Systems Engineering. p. xii–xii (2013)
29. Wieringa, R.J.: Design Science Methodology for Information Systems and Software Engineering. Springer-Verlag Berlin Heidelberg (2014)
30. Wohlin, C., Runeson, P., Höst, M., Ohlsson, M.C., Regnell, B., Wesslén, A.: Experimentation in Software Engineering. Springer Berlin Heidelberg (2012)
31. Zhang, S., Hu, Y., Bian, G.: Research on string similarity algorithm based on levenshtein distance. In: 2017 IEEE 2nd Advanced Information Technology, Electronic and Automation Control Conference (IAEAC). pp. 2247–2251 (2017)
32. Zhao, L., Alhoshan, W., Ferrari, A., Letsholo, K.J., Ajagbe, M.A., Chioasca, E.V., Batista-Navarro, R.T.: Natural language processing (nlp) for requirements engineering: A systematic mapping study. ACM Computing Surveys **54** (2021)